

Assignment 1 — Search Strategies

BFS · DFS · Greedy Best-First · A*

Dr Naila Marir

Instructor: Dr Naila Marir

Total marks: 100

Sections: 3081-1 (CRN 10480), 3081-2 (CRN 10481).

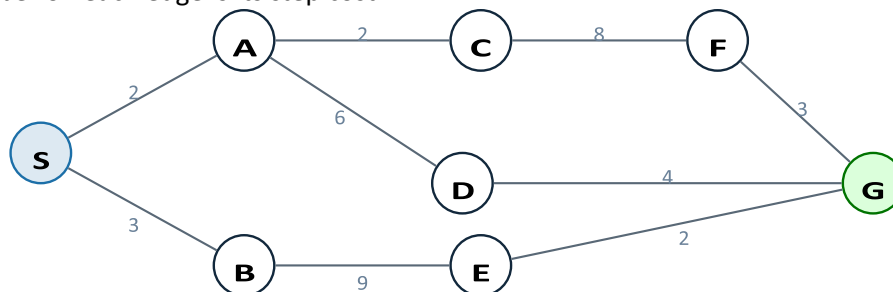
Tala Alzaeem – S26101587

Instructions. Answer all four questions. Show your working — a correct final answer with no reasoning earns partial credit only. Where a question asks for the order in which nodes are expanded, write the nodes as a sequence, e.g. *S,A,C,G*.

- When two nodes have equal priority, break ties **alphabetically**.
- A node is *expanded* when it is removed from the frontier and its successors are generated. Counting the goal test, a node is expanded even if it turns out to be the goal.
- Assume the **graph-search** form of each algorithm: never expand the same state twice.
- This is an individual assignment. You may discuss ideas, but the work you submit must be your own.

1 Uninformed search: BFS and DFS

Consider the following state graph. The start state is *S* and the goal state is *G*. Edges are undirected, and the number on each edge is its step cost.



For this question ignore the edge costs. BFS and DFS are *uninformed*: they do not look at cost or at any estimate of distance to the goal.

(a) Breadth-first search

[8 marks]

List the order in which states are expanded by BFS, and give the path it returns.

Answer: order expanded: S, A, B, C, D, E, F, G

Path returns: S, A, D, G

Answer questions 1) a, b:

Q1: • BFS "Queue" First in First out

The path is:
S, A, B, C, D, E, F, G

Expand	add	Frontier
S	Push A Push B	A, B
A	push C push D	B, C, D
B	push E	C, D, E
C	push F	D, E, F
D	push G	E, F, G
E	-	F, G
F	-	G
G	the goal!	

• DFS "Stack" First in last out

The path is:
S, A, C, F, G

Expand	add	Frontier
S	Push B Push A	A, B
A	push D push C	C, D, B
C	push F	F, D, B
F	push G	G, D, B
G	the goal!	

(b) Depth-first search**[8 marks]**

List the order in which states are expanded by DFS, and give the path it returns. Assume successors are pushed onto the stack in reverse alphabetical order, so that the alphabetically first successor is expanded next.

Answer: order expanded: S, A, C, F, G

Path returned: S, A, C, F, G

(c) Comparison**[6 marks]**

BFS and DFS returned different paths. State which path is shorter *in number of steps*, and explain why BFS is guaranteed to find that path while DFS is not.

Answer: Shorter path: The path returned by BFS is shorter in the number of steps than the DFS). Because BFS explores shallowest depth first but the DFS follows a single path as deeply as possible, meaning it returns the first valid path regardless of its total step count.

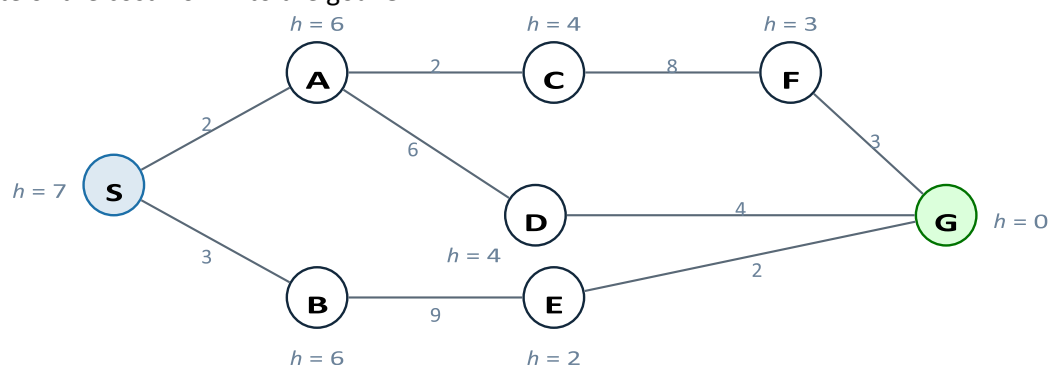
(d) Frontier data structures**[3 marks]**

BFS and DFS differ by one design choice. Name the data structure each uses for its frontier, and explain in one sentence how that choice produces the difference in behavior you observed.

Answer: BFS uses a (First-In, First-Out) 'Queue', while DFS uses a (First-in, Last-out) 'Stack'. The queue ensures that nodes are expanded in the exact order they are discovered, producing a shallow, level-by-level breadth exploration, whereas a stack forces the algorithm to prioritize and expand the most recently generated successors first, creating a deep, single-branch exploration.

2 Informed search: Greedy Best-First and A*

The same graph is repeated below. Each state is now annotated with a heuristic value $h(n)$ — an estimate of the cost from n to the goal G .



Answer questions 2) a, b:

Q2: Greedy Best-First

Expand	add	Frontier
S (h=7)	Push A (h=6) Push B (h=6)	A, B
A (h=6)	Push D (h=4) Push C (h=4)	C, D, B
C (h=4)	Push F (h=3)	F, D, B
F (h=3)	push G (h=0)	G, D, B
G (h=0)	The goal!	

The path: S, A, C, F, G

A* search

order	NODE	$g(n)$	$h(n)$	$F(n) = g(n) + h(n)$
1	S	0	7	7
2	A	2	6	8
3	C	4	4	8
4	B	3	6	9
5	D	8	4	12
6	G	12	0	12

(a) Greedy best-first search**[8 marks]**

Greedy best-first expands the frontier node with the smallest $h(n)$. List the order of expansion and the path returned.

Answer: order expanded S, A, C, F, G

Path returned: S, A, C, F, G

(b) A* search**[12 marks]**

A* expands the frontier node with the smallest $f(n) = g(n) + h(n)$. Complete the table for every node *at the moment it is expanded*, then give the order of expansion and the path returned.

Order	Node	$g(n)$	$h(n)$	$f(n) = g(n) + h(n)$
1	S	0	7	7
2	A	2	6	8
3	C	4	4	8
4	B	3	6	9
5	D	8	4	12
6	G	12	0	12

Order of expansion:

S, A, C, B, D, G

Path returned:

S, A, D, G

Total cost:

12

(c) Why they differ**[6 marks]**

Greedy best-first and A* returned different paths, or explored a different number of nodes, or both. Explain the difference by referring to what each algorithm ignores.

Answer: Greedy best-first search ignores the distance already traveled $g(n)$. It only looks at how close a node seems to be to the goal $h(n)$. Because it ignores the past cost, it can easily choose a path that ends up being way too long or expensive.

A* search ignores neither. It looks at both the past cost $g(n)$ and the estimated future cost $h(n)$. Because it accounts for both, it can spot when a path is getting too expensive and find the true shortest route.

3 Admissibility and optimality**(a) Definition****[5 marks]**

Define an **admissible** heuristic. Your definition must be precise enough that someone could check a given heuristic against it.

Answer: An admissible heuristic is a heuristic function $h(n)$ that never overestimates the true cost to reach the goal state from any given node n . To check a heuristic precisely against this definition, it must satisfy the mathematical condition for every node n in the search space: $0 \leq h(n) \leq h^*(n)$

(b) Testing the heuristic**[10 marks]**

For the graph in Question 2, compute the true cost $h^*(n)$ of the cheapest path from each node to G, and decide whether the given h is admissible.

Node	S	A	B	C	D	E	F	G
$h(n)$ given	7	6	6	4	4	2	3	0
$h^*(n)$ true	<u>12</u>	<u>10</u>	<u>11</u>	<u>11</u>	<u>4</u>	<u>2</u>	<u>3</u>	<u>0</u>
$h \leq h^*$?	<u>yes</u>	<u>yes</u>	<u>yes</u>	<u>yes</u>	<u>yes</u>	<u>yes</u>	<u>yes</u>	<u>yes</u>

Is h admissible? yes

If not, name every node that violates the condition: —

(c) Consequence**[6 marks]**

Suppose a heuristic overestimate the true cost at exactly one node. Explain what this can do to the solution A^* returns, and why BFS could never suffer from this problem.

Answer: A Search:* If a heuristic overestimates the cost, A^* can mistakenly think a good path is too expensive. As a result, it may skip the shortest path entirely and return a suboptimal (worse) solution.

BFS: BFS does not use heuristics to guide its search. It only looks at the number of steps taken, so an overestimating heuristic cannot change its behavior.

(d) A trivial heuristic**[3 marks]**

What does A^* reduce to when $h(n) = 0$ for every node? Justify your answer in one sentence.

Answer: When $h(n) = 0$ for every node, A^* search reduces to Uniform Cost Search.

Justification: Since the evaluation function becomes $f(n) = g(n) + 0 = g(n)$, the algorithm chooses nodes based solely on their actual path cost from the start node.

4 Comparing the four strategies

(a) Properties table

[8 marks]

Complete the table. Name the data structure each strategy uses for its frontier. For *optimal*, write Yes or No and state any condition required for that answer to hold.

Strategy	Frontier	Optimal?
Breadth-first	Queue (first in first out).	Yes, if all step costs are identical/equal.
Depth-first	Stack (first in last out).	No
Greedy best-fir	ordered by heuristic value $h(n)$.	No
A*	ordered by total estimated cost $g(n) + h(n)$.	Yes, if $h(n)$ is admissible (never overestimates the true cost) and the $h(n)$ is consistent (for every node n and successor n' with step cost c , $h(n) \leq h(n') + c$)

(b) Choosing a strategy

[13 marks]

For each scenario, name the most appropriate of the four strategies and justify your choice. Refer to the properties in part (a) — a bare name earns no marks.

- i. A robot must find *any* route out of a building during a fire. Memory is very limited; the route need not be shortest. **DFS (Depth-First Search)** Justification: DFS uses much less memory because it mainly stores the current path and unexplored branches. It can find any solution without requiring to be the shortest path.
- ii. A route planner must return the *cheapest* driving route, and a good straight-line-distance estimate is available. **A* search** Justification: The goal requires finding the cheapest driving route (optimality), and a good straight-line-distance estimate (heuristic function) is available. A* search efficiently combines the actual path cost from the start state with an admissible heuristic estimate to guarantee an optimal path while minimizing the number of explored nodes, $f(n)=g(n)+h(n)$.
- i. A puzzle has uniform step costs, no useful heuristic, and the solution is known to be shallow. **BFS (Breadth-First Search)** Justification: BFS is complete and optimal when all step costs are equal, because it explores nodes level by level. Since the solution is known to be shallow, BFS is particularly suitable because it will find the shallowest solution quickly.

(c) Short answer**[4 marks]**

A* is optimal but is often impractical on large problems. State the reason, and name one property of the search that makes it worse.

Answer: A* is often impractical because it uses too much computer memory (RAM) and easily crashes large programs.

The main property that makes it worse is that it must remember every single path it explores. It saves every location it visits in a massive list so it doesn't get lost or repeat steps. For big problems, this list grows so quickly that the computer completely runs out of memory before finding the answer.

Submission. Submit a single PDF to Blackboard by the due date. Handwritten work is acceptable if scanned clearly. Include your name and student ID on the first page.

Q1: 25 Q2: 26 Q3: 24 Q4: 25 **Total: 100 marks**